



Lab #2: Data Segment, Loops and Simple Programs

Objective

- Learn to use full segment definition and simple segment definition
- Learn to use directives in assembly

Directives to define segments

In x86 assembly language, directives like `.data`, `.code`, and `.stack` are used to define different memory segments. The `.data` segment is where initialized variables are declared, such as constants or global data that the program will use. The `.code` segment (sometimes `.text` in some assemblers) contains the actual program instructions or executable code. Finally, the `.stack` segment is used to define the stack, a special area of memory for function calls, local variables, and control flow management. These segments help structure the program, allowing the assembler to allocate and manage memory efficiently during execution.

Labels

In assembly language, labels are markers used to identify specific locations in a program, such as instructions or data. They serve as reference points that can be used for jumps, loops, or data access. A code label is typically followed by a colon and is placed before an instruction.

Label Type	Description
Code Label	Marks a specific point in the instruction flow, often used for jumps or function calls or loops.
Data Label	Marks the location of data or variables in memory, used to reference variables.

Sample use of labels:

```
.data
num1 WORD 10          ; Data label for a variable

.code
main PROC
start:                ; Code label marking the entry point
    mov ax, num1      ; Referencing data label in code
exit
main ENDP
```

Data Types

In x86 assembly language, intrinsic data types represent different sizes of data that the processor can handle. These data types correspond to the various ways memory is organized and accessed by the CPU.

The most common data types include bytes, words, doublewords, and quadwords, which are used to define variables or manipulate memory in different capacities.

Data Type	Size (bits)	Size (bytes)	Memory Blocks	Description
BYTE	8 bits	1 byte	1 block	8-bit unsigned integer/character (similar to C++ char)
SBYTE	8 bits	1 byte	1 block	8-bit signed integer (similar to C++ signed char)
WORD	16 bits	2 bytes	2 blocks	16-bit unsigned integer (similar to C++ unsigned short)
SWORD	16 bits	2 bytes	2 blocks	16-bit signed integer (similar to C++ short)
DWORD	32 bits	4 bytes	4 blocks	32-bit unsigned integer (similar to C++ unsigned int)
SDWORD	32 bits	4 bytes	4 blocks	32-bit signed integer (similar to C++ int)
QWORD	64 bits	8 bytes	8 blocks	64-bit integer (similar to C++ long long)
TBYTE	80 bits	10 bytes	10 blocks	80-bit integer (used for extended precision)
REAL4	32 bits	4 bytes	4 blocks	4-byte IEEE short real (similar to C++ float)
REAL8	64 bits	8 bytes	8 blocks	8-byte IEEE long real (similar to C++ double)
REAL10	80 bits	10 bytes	10 blocks	10-byte IEEE extended real (similar to C++ long double)

Loop Instruction

The LOOP instruction in x86 assembly is used to implement loops by decrementing the **ECX** register and repeating the execution of a block of code until **ECX reaches zero**. The LOOP instruction automatically decrements ECX by 1 and checks if it's zero. If it is not zero, the instruction jumps to a specified label to repeat the loop. If ECX becomes zero, the loop exits, and execution continues with the next instruction. This instruction is useful for creating counted loops without needing to manually compare or jump.

Sample use of Loop:

```
INCLUDE Irvine32.inc

.data
C DWORD 10
.code
main PROC
    mov ecx, C                ; Set the loop counter to 5

count_loop:
    ; Convert and print the current value of ECX
    mov eax, ecx              ; Move counter value to EAX
    call WriteInt; Convert and print number

    ; Print space after each number
    mov al, ' '                ; Load space character
    call WriteChar             ; Print space

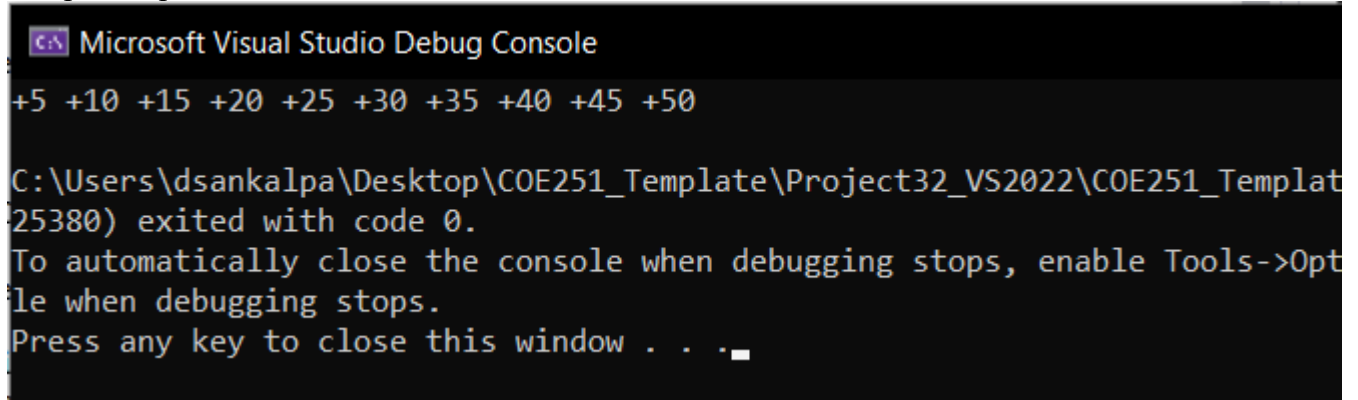
    loop count_loop            ; Decrement ECX and loop if not zero
    call clr                  ; carriage return and line feed

    exit                      ; Exit the program
main ENDP
```

Lab Work:

1. Write a program that prints the first 10 multiples of 5, each followed by a space. Use a loop and appropriate arithmetic operations to generate the multiples.

Sample Output:



```
C:\Users\dsankalpa\Desktop\COE251_Template\Project32_VS2022\COE251_Template\Project32_VS2022\COE251_Template.exe (25380) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
; Your data goes here
value DWORD 5; initialize value to 5
```

.code

main PROC

```
; Your code goes here
mov ecx, 10 ;ecx is used to control LOOP
```

again:

```
mov eax, value ;std::cout << value;
call WriteInt
```

```
mov al, ' ' ;std::cout << ' ';
call WriteChar
```

```
mov eax, value ;value += value;
add eax, 5
mov value, eax
```

```
loop again ;loops code (loops for 10 times as specified in ecx)
```

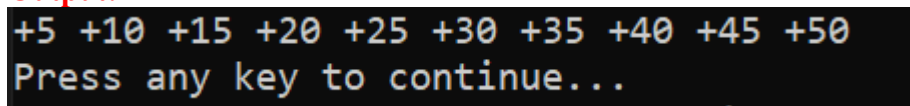
```
call crlf ;carriage return and line feed
call WaitMsg ;Pauses program execution until the user presses a key
```

exit

main ENDP

END main

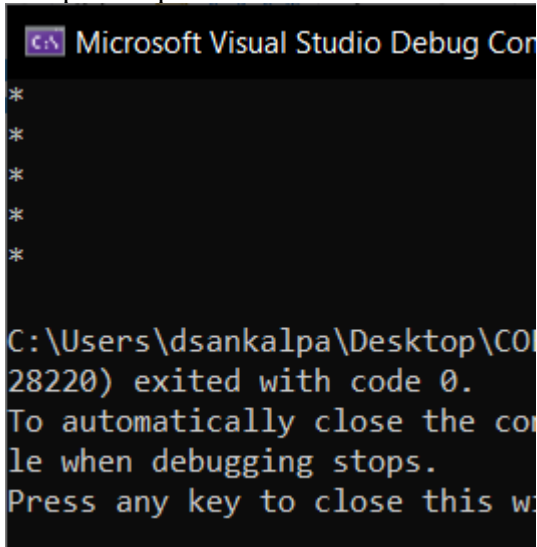
Output:



```
+5 +10 +15 +20 +25 +30 +35 +40 +45 +50
Press any key to continue...
```

2. Write an assembly language program that prints a vertical line of stars (*) on the console. The vertical line should consist of 5 stars, each on a new line.

Sample Output:



Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

 ; Your data goes here
 value DWORD "*"; initialzie value to 3

.code

main PROC

 ; Your code goes here
 mov ecx, 5 ;ecx is used to control LOOP

again:

 mov eax, value ;std::cout << value;
 call WriteChar

 call crlf ;carriage return and line feed

 loop again ;loops code (loops for 10 times as specified in ecx)

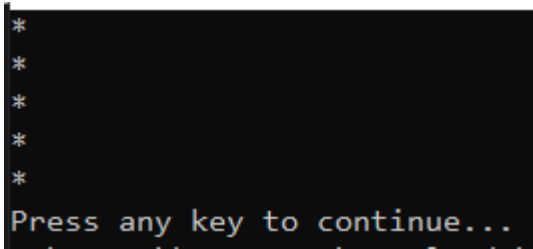
 call WaitMsg ;Pauses program execution until the user presses a key

 exit

main ENDP

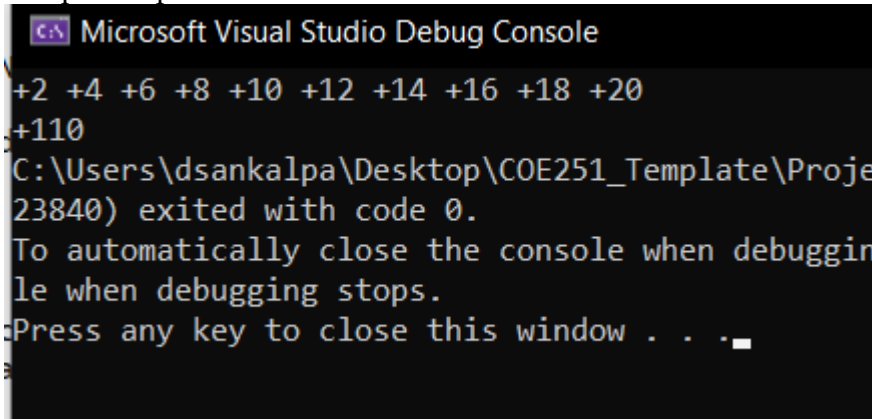
END main

Output:



- Write an assembly program to calculate the sum of all even numbers in between of 0 and 20 inclusive. The sum must be stored in a data variable called even_sum and must be displayed to the user.

Sample Output:



Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
; Your data goes here
value DWORD 2; initialzie value to 3
even_sum DWORD 0
```

.code

main PROC

```
; Your code goes here
mov ecx, 10 ;ecx is used to control LOOP
```

again:

```
mov eax, value ;std::cout << value;
call WriteInt
add even_sum, eax
```

```
mov al, ' ' ;std::cout << ' ';
call WriteChar
```

```
add value, 2
```

```
loop again ;loops code (loops for 10 times as specified in ecx)
```

```
call crlf ;carriage return and line feed
```

```

mov eax, even_sum
call WriteInt
call crlf

call WaitMsg          ;Pauses program execution until the user presses a key

exit
main ENDP
END main

```

Output:

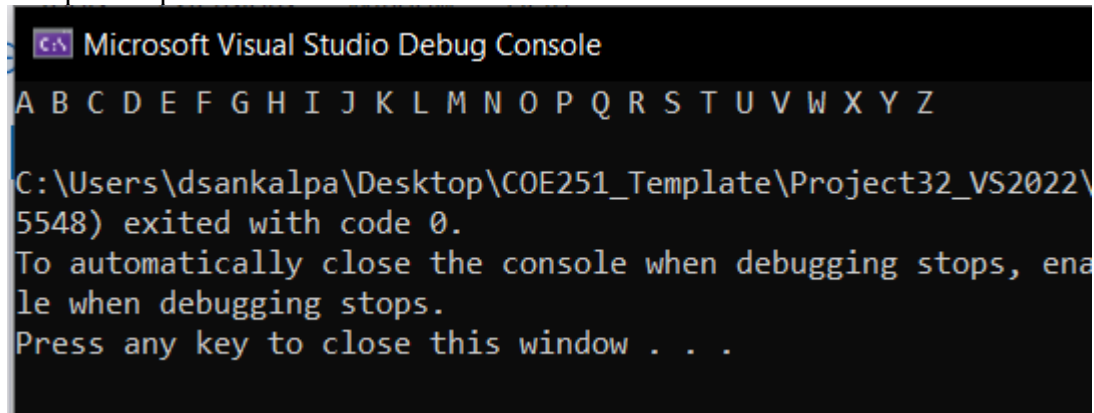
```

+2 +4 +6 +8 +10 +12 +14 +16 +18 +20
+110

```

4. Write an assembly language program that prints all the uppercase letters of the English alphabet (A to Z) on a single line, with each letter separated by a space.

Sample Output:



```

Microsoft Visual Studio Debug Console
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
C:\Users\dsankalpa\Desktop\C0E251_Template\Project32_VS2022\
5548) exited with code 0.
To automatically close the console when debugging stops, ena
le when debugging stops.
Press any key to close this window . . .

```

Solution:

```

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data
    ; Your data goes here
    value DWORD 65; initialzie value to 3

.code
main PROC
    ; Your code goes here
    mov ecx, 26          ;ecx is used to control LOOP

again:
    mov eax, value        ;std::cout << value;
    call WriteChar

    mov al, ' '           ;std::cout << ' ';
    call WriteChar

```



```

mov  eax, value                ;value += value;
add  eax, eax
mov  value, eax

loop again                    ;loops code (loops for 10 times as specified in ecx)

call crlf                    ;carriage return and line feed
call WaitMsg                 ;Pauses program execution until the user presses a key

exit
main ENDP
END main

```

Output:

```

+1 +2 +4 +8 +16 +32 +64 +128 +256 +512 +1024 +2048 +4096 +8192 +16384
Press any key to continue...

```